```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

```
df = pd.read_csv("heart_disease.csv")
df.head()
```

	age	sex	cholesterol	blood_pressure	max_heart_rate	exercise_angina	heart_disease
0	67	0	184	99	154	0	0
1	57	1	198	162	192	1	1
2	43	0	166	113	161	0	0
3	71	1	307	153	62	0	1
4	36	0	195	138	82	0	0

```
df.shape
```

```
(200, 7)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 7 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   age              200 non-null    int64
 1   sex              200 non-null    int64
 2   cholesterol      200 non-null    int64
 3   blood_pressure   200 non-null    int64
 4   max_heart_rate   200 non-null    int64
 5   exercise_angina  200 non-null    int64
 6   heart_disease    200 non-null    int64
dtypes: int64(7)
memory usage: 11.1 KB
```

```
df.isnull().sum()
```

	0
age	0
sex	0
cholesterol	0
blood_pressure	0
max_heart_rate	0
exercise_angina	0
heart_disease	0

dtype: int64

```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
df["sex"] = le.fit_transform(df["sex"])
```

```
df.head()
```

	age	sex	cholesterol	blood_pressure	max_heart_rate	exercise_angina	heart_disease
0	67	0	184	99	154	0	0
1	57	1	198	162	192	1	1
2	43	0	166	113	161	0	0
3	71	1	307	153	62	0	1
4	36	0	195	138	82	0	0

```
df.columns
```

```
Index(['age', 'sex', 'cholesterol', 'blood_pressure', 'max_heart_rate',
       'exercise_angina', 'heart_disease'],
      dtype='object')
```

```
# Setting the target and Feature Variables
```

```
x = df[['age', 'sex', 'cholesterol', 'blood_pressure', 'max_heart_rate',
       'exercise_angina']]
y = df['heart_disease']
```

```
from sklearn.feature_selection import SelectKBest,f_classif
```

```
skb = SelectKBest(f_classif, k =5)
select = skb.fit_transform(x,y)
column_name = x.columns[skb.get_support()]
column_name
```

```
Index(['age', 'sex', 'cholesterol', 'max_heart_rate', 'exercise_angina'], dtype='object')
```

```
x_new = ['age', 'sex', 'cholesterol', 'max_heart_rate', 'exercise_angina']
```

```
from sklearn.model_selection import train_test_split
```

```
xtrain, xtest, ytrain, ytest = train_test_split(x, y, test_size = 0.2, random_state = 42)
xtrain.shape, xtest.shape, ytrain.shape, ytest.shape
```

```
((160, 6), (40, 6), (160,), (40,))
```

```
from sklearn.preprocessing import StandardScaler
```

```
SS = StandardScaler()
xtrain = SS.fit_transform(xtrain)
xtest = SS.fit_transform(xtest)
```

```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
```

```
model = Sequential()

model.add(Dense(32, activation = "relu", input_shape = (xtrain.shape[1],)))
model.add(Dense(16, activation = "relu"))
model.add(Dense(1, activation = "sigmoid"))
```

```
/usr/local/lib/python3.13/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
# Compiling the model
model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])
```

```
from tensorflow.keras.callbacks import EarlyStopping
```

```
early_stopping = EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True)

# Training the model
```

```
# Training the model
history = model.fit(
    xtrain, ytrain,
    batch_size=32,
    epochs=500,
    validation_split=0.2,
    callbacks=[early_stopping]
)
```

```
Epoch 1/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 1s 57ms/step - accuracy: 0.4375 - loss: 0.7134 - val_accuracy: 0.7500 - val_loss: 0.6506
Epoch 2/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.4609 - loss: 0.7078 - val_accuracy: 0.7500 - val_loss: 0.6518
Epoch 3/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - accuracy: 0.4609 - loss: 0.7037 - val_accuracy: 0.7500 - val_loss: 0.6532
Epoch 4/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.4844 - loss: 0.6997 - val_accuracy: 0.7812 - val_loss: 0.6552
Epoch 5/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.4922 - loss: 0.6959 - val_accuracy: 0.7500 - val_loss: 0.6568
Epoch 6/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.4922 - loss: 0.6928 - val_accuracy: 0.7812 - val_loss: 0.6588
Epoch 7/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.5000 - loss: 0.6893 - val_accuracy: 0.7812 - val_loss: 0.6604
Epoch 8/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - accuracy: 0.5078 - loss: 0.6864 - val_accuracy: 0.7812 - val_loss: 0.6618
Epoch 9/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.5078 - loss: 0.6834 - val_accuracy: 0.7812 - val_loss: 0.6636
Epoch 10/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.5078 - loss: 0.6809 - val_accuracy: 0.7812 - val_loss: 0.6652
Epoch 11/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.5234 - loss: 0.6783 - val_accuracy: 0.6875 - val_loss: 0.6673
```

```
# Evaluating the model
loss, accuracy = model.evaluate(xtest, ytest)
print("Accuracy", accuracy)
print("Loss", loss)
```

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.6000 - loss: 0.6952
Accuracy 0.6000000238418579
Loss 0.6951514482498169
```

```
# Making predictions
y_pred = model.predict(xtest)
y_pred[0:5]
```

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 39ms/step
array([[0.5259491 ],
       [0.34074715],
       [0.3866828 ],
       [0.3615712 ],
       [0.4136747 ]], dtype=float32)
```

```
# Converting from Probability
y_pred = (y_pred > 0.5).astype(int)
y_pred[0:5]
```

```
array([[1],
       [0],
       [0],
       [0],
       [0]])
```

```
y_compare = pd.DataFrame(ytest)
y_compare['Predicted'] = y_pred
y_compare = y_compare.reset_index()
y_compare.drop('index', axis = 1)
```

	heart_disease	Predicted
0	1	1
1	1	0
2	0	0
3	1	0
4	1	0
5	0	0
6	1	1
7	0	1
8	0	0
9	0	0
10	0	1
11	1	0
12	0	0
13	0	0
14	1	1
15	0	0
16	0	0
17	0	0
18	1	0
19	1	0
20	0	0
21	0	0
22	1	0
23	1	1
24	1	0
25	0	1
26	1	0
27	0	0
28	0	1
29	1	0
30	1	0
31	0	0
32	1	1
33	1	1
34	0	0
35	1	0
36	0	0
37	0	0
38	0	0
39	0	0

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

```
cr = classification_report(ytest, y_pred)
print(cr)
```

```
              precision    recall  f1-score   support
```

```
              0       0.60      0.82      0.69        22
              1       0.60      0.33      0.43        18

       accuracy                           0.60        40
      macro avg       0.60      0.58      0.56        40
   weighted avg       0.60      0.60      0.57        40
```